



Plataforma Multi-Agente de Validação Estratégica de Negócios

Documento de Especificação de Requisitos de Software — DERS

Disciplina	ESW308 — Tópicos Especiais em Software
Aluno	Samuel Guimarães Lopes
Versão	1.0
Data	28/05/2026
Status	Em desenvolvimento

Histórico de Revisões

Versão	Data	Autor	Descrição
1.0	28/05/2026	Samuel Guimarães Lopes	Versão inicial do documento

1. Introdução

1.1 Objetivo do Documento

Este documento descreve os requisitos de software do sistema **PitchIQ**, uma plataforma multi-agente de validação estratégica de ideias de negócio. O DERS apresenta os requisitos funcionais, não-funcionais, restrições e casos de uso que guiarão o desenvolvimento e avaliação do produto ao longo do semestre.

1.2 Escopo do Sistema

O PitchIQ é uma aplicação web que recebe uma descrição textual de uma ideia de negócio e aciona autonomamente uma equipe de **9 agentes de inteligência artificial** para pesquisar, debater e sintetizar uma análise estratégica completa. O sistema produz relatórios executivos com análise de mercado, mapeamento de concorrentes, perfil de cliente, debate adversarial (Advocate vs. Devil's Advocate) e score de viabilidade.

O sistema **NÃO** se propõe a: fornecer aconselhamento financeiro formal, garantir precisão absoluta dos dados de mercado, substituir due diligence profissional ou gerenciar implementação de negócios.

1.3 Definições, Acrônimos e Abreviações

Termo	Definição
Agente de IA	Sistema autônomo baseado em LLM que executa tarefas com uso de ferramentas
LLM	Large Language Model — modelo de linguagem de grande escala (ex: Claude)
Tool Use	Capacidade do agente de invocar ferramentas externas (busca web, APIs)
SSE	Server-Sent Events — protocolo de streaming do servidor para o cliente
ICP	Ideal Customer Profile — perfil do cliente ideal
TAM/SAM/SOM	Total/Serviceable/Obtainable Addressable Market
SWOT	Strengths, Weaknesses, Opportunities, Threats
Debate Mode	Módulo com Advocate, Devil's Advocate e Mediator Agents
Pipeline	Sequência orquestrada de execução dos agentes
FastAPI	Framework Python para construção de APIs assíncronas
Next.js	Framework React para aplicações web com renderização híbrida
Tavily	API de busca web otimizada para agentes de IA

1.4 Referências

- Anthropic API Documentation — <https://docs.anthropic.com>
- Tavily Search API — <https://docs.tavily.com>

- FastAPI Documentation — <https://fastapi.tiangolo.com>
- Next.js 14 Documentation — <https://nextjs.org/docs>
- IEEE 830-1998 — Software Requirements Specifications

2. Descrição Geral do Sistema

2.1 Perspectiva do Produto

O PitchIQ é um produto independente desenvolvido como projeto final da disciplina ESW308. O sistema integra a API da Anthropic (Claude Sonnet) como motor de raciocínio dos agentes e a API Tavily para busca web em tempo real. A arquitetura segue o padrão cliente-servidor com frontend em Next.js e backend em Python/FastAPI.

2.2 Arquitetura Multi-Agente

O núcleo do sistema é composto por 9 agentes especializados organizados em 3 fases:

Fase	Agente	Responsabilidade
1 — Pesquisa (paralela)	Market Agent Competitor Agent Audience Agent Risk Agent	Pesquisa mercado (TAM/SAM/SOM) Mapeia concorrentes e gaps Define ICP e canais Avalia riscos regulatórios e de execução
2 — Debate	Advocate Agent Devil's Advocate Mediator Agent	Defende a ideia com dados Desafia premissas e riscos Arbitragem e veredicto balanceado
3 — Síntese	Synthesis Agent Q&A Agent	Consolida relatório executivo + score Responde perguntas sobre o relatório

2.3 Funções Principais do Produto

- Receber descrição textual de uma ideia de negócio em linguagem natural
- Orquestrar pipeline de 9 agentes de IA em fases sequenciais e paralelas
- Realizar busca web em tempo real via Tavily para embasar a análise
- Executar debate adversarial automatizado (Advocate vs. Devil's Advocate)
- Gerar relatório executivo estruturado com 9 seções e score de viabilidade
- Exibir progresso dos agentes em tempo real via Server-Sent Events (SSE)
- Permitir consultas (Q&A;) ao relatório gerado via agente especializado

2.4 Características dos Usuários

Perfil	Empreendedores, estudantes, profissionais de produto e estratégia
Conhecimento técnico	Nenhum — interface intuitiva sem jargão técnico
Frequência de uso	Pontual (1-5 análises por sessão)
Dispositivo	Desktop/notebook (interface otimizada para telas >= 1024px)

2.5 Restrições Gerais

- Requer conexão com a internet para acesso às APIs externas (Anthropic + Tavily)
- Chaves de API válidas e com crédito disponível são obrigatórias para funcionamento
- O tempo de análise varia entre 1 e 5 minutos dependendo da complexidade da ideia
- A sessão de análise é mantida em memória (não persistida entre reinicializações do servidor)
- Limite de taxa da API Anthropic: 30.000 tokens/minuto no plano inicial

2.6 Suposições e Dependências

O sistema depende de: (a) disponibilidade das APIs externas (Anthropic e Tavily), (b) ambiente Python 3.11+ com dependências instaladas, (c) ambiente Node.js 18+ para o frontend. Assume-se que o usuário possui navegador moderno com suporte a Server-Sent Events.

3. Requisitos Funcionais

Os requisitos funcionais descrevem o comportamento esperado do sistema. Prioridades: **Alta** = essencial para o MVP, **Média** = importante mas não bloqueante, **Baixa** = desejável para versões futuras.

3.1 Entrada e Validação da Ideia

ID	Nome	Descrição	Prioridade
RF001	Entrada de ideia	O sistema deve aceitar ideias de negócio em texto livre (mínimo 10, máximo 1000 caracteres)	Alta
RF002	Validação de entrada	O sistema deve rejeitar entradas vazias ou muito curtas com mensagem de erro explicativa.	Alta
RF003	Exemplos pré-carregados	O sistema deve oferecer ao menos 4 exemplos de ideias clicáveis que preencham o campo	Média

3.2 Orquestração do Pipeline Multi-Agente

ID	Nome	Descrição	Prioridade
RF004	Disparo do pipeline	Ao submeter uma ideia, o sistema deve iniciar automaticamente o pipeline de 9 agentes serialmente	Alta
RF005	Execução paralela	Os 4 agentes da Fase 1 (Market, Competitor, Audience, Risk) devem ser executados em paralelo	Alta
RF006	Execução sequencial	A Fase 2 (Debate) deve iniciar somente após a conclusão de todos os agentes da Fase 1. A Fase 3 é executada em paralelo	Alta
RF007	Identificação de sessão	O sistema deve gerar e retornar um ID único de sessão para cada análise, permitindo recuperação de estado	Alta
RF008	Retry automático	O sistema deve realizar até 5 tentativas com backoff exponencial em caso de erros de rate limit	Média

3.3 Agentes de Pesquisa (Fase 1)

ID	Nome	Descrição	Prioridade
RF009	Market Agent	Deve realizar ao menos 2 buscas web e retornar análise de TAM/SAM/SOM, tendências do mercado	Média
RF010	Competitor Agent	Deve identificar ao menos 3 concorrentes reais (nacionais ou internacionais), com forças, fraquezas	Alta
RF011	Audience Agent	Deve definir o ICP com perfil demográfico, dores, comportamentos e de compra, canais de aquisição	Alta
RF012	Risk Agent	Deve mapear riscos regulatórios, de mercado e de execução, cada um com avaliação de probabilidade	Alta
RF013	Busca web real	Os agentes de pesquisa devem usar a ferramenta Tavily para buscar informações atualizadas	Baixa

3.4 Debate Adversarial (Fase 2)

ID	Nome	Descrição	Prioridade
RF014	Advocate Agent	Deve apresentar os argumentos mais fortes a favor da ideia, usando dados da Fase 1, em formato de tópicos	Alta
RF015	Devil's Advocate	Deve contra-argumentar diretamente os pontos do Advocate, identificando premissas não validadas	Alta
RF016	Mediator Agent	Deve mediar o debate, reconhecer pontos válidos de ambos os lados, definir as 3 principais conclusões	Alta

RF017	Contexto compartilhado	O Devil's Advocate deve receber como input o argumento do Advocate. O Mediator deve re	Alta
-------	------------------------	--	------

3.5 Síntese e Relatório (Fase 3)

ID	Nome	Descrição	Prioridade
RF018	Relatório executivo	O Synthesis Agent deve consolidar todas as análises em relatório com: Resumo Executivo,	Alta
RF019	Score de viabilidade	O relatório deve incluir uma nota global de 0-100 e scores individuais para: mercado, compe	Alta
RF020	Score em JSON	O Synthesis Agent deve emitir os scores em bloco JSON padronizado ao final do relatório p	Alta
RF021	Q&A Agent	Após a conclusão da análise, o usuário deve poder fazer perguntas em linguagem natural e	Alta
RF022	Anti-alucinação Q&A	O Q&A Agent deve declarar explicitamente quando uma informação não foi coberta na análise	Alta

3.6 Interface e Streaming

ID	Nome	Descrição	Prioridade
RF023	Streaming em tempo real	O frontend deve exibir o status de cada agente (idle, running, down) em tempo real via SSE	Alta
RF024	AgentTimeline	A interface deve exibir uma linha do tempo visual com os 8 agentes do pipeline, indicando a	Alta
RF025	DebateView	O frontend deve apresentar o debate com identidade visual distinta para cada agente (cores	Alta
RF026	ScoreRadar	O frontend deve exibir os scores por dimensão em um gráfico interativo.	Média
RF027	Abas de resultado	A interface deve organizar os resultados em abas: Relatório, Debate e Score.	Alta
RF028	Cópia do relatório	O usuário deve poder copiar o relatório completo para a área de transferência com um cliqu	Média

4. Requisitos Não-Funcionais

4.1 Desempenho

ID	Nome	Descrição	Prioridade
RNF001	Tempo total de análise	O pipeline completo (9 agentes) deve concluir em no máximo 5 minutos em condições normais.	Alta
RNF002	Paralelismo	A Fase 1 deve processar os 4 agentes simultaneamente, reduzindo o tempo em relação à execução sequencial.	Alta
RNF003	Latência do streaming	O frontend deve exibir a mudança de status de cada agente em no máximo 2 segundos após a mudança no backend.	Média
RNF004	Resposta do Q&A	O Q&A Agent deve retornar resposta em no máximo 30 segundos após a pergunta.	Média

4.2 Usabilidade

ID	Nome	Descrição	Prioridade
RNF005	Onboarding zero	Um usuário sem conhecimento técnico deve conseguir submeter uma ideia e interpretar o resultado.	Alta
RNF006	Feedback visual	O sistema deve sempre informar visualmente o estado do processo (loading, running, erro).	Alta
RNF007	Mensagens de erro claras	Em caso de falha, o sistema deve exibir mensagem em português explicando o problema e sugerindo ações.	Alta
RNF008	Responsividade	A interface deve ser utilizável em telas a partir de 1024px de largura.	Média

4.3 Confiabilidade

ID	Nome	Descrição	Prioridade
RNF009	Retry automático	O sistema deve recuperar-se automaticamente de erros temporários de API (rate limit, timeout).	Alta
RNF010	Isolamento de falhas	A falha de um agente não deve interromper os demais agentes em execução paralela.	Alta
RNF011	Disponibilidade local	O sistema deve funcionar no ambiente local sem dependências de infraestrutura em nuvem.	Alta

4.4 Segurança

ID	Nome	Descrição	Prioridade
RNF012	Proteção de chaves	Chaves de API devem ser armazenadas em variáveis de ambiente (.env) e nunca expostas no código.	Alta
RNF013	CORS configurado	O backend deve aceitar requisições apenas de origens autorizadas (localhost:3000 e domínio de produção).	Alta
RNF014	Dados em memória	Sessões de análise são armazenadas apenas em memória do processo, sem persistência em disco.	Média

4.5 Manutenibilidade

ID	Nome	Descrição	Prioridade
RNF015	Modularidade dos agentes	Cada agente deve ser implementado em módulo Python independente, permitindo substituição sem afetar o resto do sistema.	Alta

RNF016	Sem framework de agentes	A orquestração deve usar Claude Tool Use nativo, sem dependência de LangChain ou CrewAI	Média
RNF017	Documentação inline	Funções críticas do pipeline devem ter docstrings descritivos.	Baixa

5. Casos de Uso

Os casos de uso descrevem as interações entre os atores e o sistema. O ator principal é o **Usuário** (empreendedor, estudante ou profissional). O ator secundário é o **Sistema de Agentes** (execução autônoma do pipeline).

5.1 Diagrama de Casos de Uso (textual)

ID	Caso de Uso	Descrição	Atores
UC001	Submeter ideia de negócio	O usuário digita ou seleciona uma ideia de negócio e solicita análise.	Usuário
UC002	Acompanhar pipeline ao vivo	O usuário visualiza o progresso dos agentes em tempo real.	Usuário e Agentes
UC003	Visualizar relatório	O usuário acessa o relatório executivo completo gerado pelo Synthesis Agent.	Synthesis Agent.
UC004	Explorar debate estratégico	O usuário lê a transcrição do debate entre Advocate, Devil's Advocate e Mediator.	Advocate e Mediator.
UC005	Consultar score de viabilidade	O usuário visualiza o score global e por dimensão no gráfico.	Usuário
UC006	Fazer pergunta ao Q&A Agent	O usuário digita uma pergunta sobre o relatório e recebe resposta.	Usuário/ Q&A Agent.
UC007	Copiar relatório	O usuário copia o conteúdo completo do relatório para a área de transferência.	Usuário
UC008	Iniciar nova análise	O usuário retorna à tela inicial para analisar uma nova ideia.	Usuário

5.2 Fluxo Principal — UC001: Submeter Ideia de Negócio

Pré-condição	Backend e frontend em execução. Chaves de API configuradas.
Ator	Usuário
Fluxo básico	1. Usuário acessa http://localhost:3000 2. Usuário digita a ideia de negócio no campo de texto 3. Usuário clica em 'Analisar ideia' 4. Sistema redireciona para página de análise com SSE 5. Sistema retorna session_id e inicia pipeline 6. Interface exhibe AgentTimeline com agentes 'running' 7. Cada agente conclui e emite evento 'done' 8. Pipeline emite evento 'complete' 9. Interface exhibe relatório completo com abas
Fluxo alternativo	3a. Usuário clica em um dos exemplos pré-carregados → campo preenchido automaticamente
Fluxo de exceção	5a. Falha de conexão com API → mensagem de erro exibida → usuário pode tentar novamente
Pós-condição	Relatório disponível nas abas. Q&A; Agent ativo para consultas.

6. Stack Técnica e Arquitetura

6.1 Tecnologias Utilizadas

Camada	Tecnologia	Versão	Função
LLM	Claude Sonnet (Anthropic)	4.6	Motor de raciocínio de todos os agentes
Backend	Python / FastAPI	3.13 / 0.115	API REST + SSE streaming
Busca web	Tavily API	latest	Pesquisa em tempo real para agentes
Frontend	Next.js / React	16 / 19	Interface web com streaming ao vivo
Estilo	Tailwind CSS	4.x	Design system utility-first
Gráficos	Recharts	2.x	Gráfico radar do score de viabilidade
Env vars	python-dotenv	1.x	Gestão de chaves de API

6.2 Estrutura de Diretórios

```
pitchiq/ █ █ █ █ backend/ █ █ █ █ main.py # FastAPI app (endpoints /analyze, /qa, /session) █ █ █ █ agents/ █ █ █ █ base.py # Cliente Anthropic + retry logic + call_llm() █ █ █ █ orchestrator.py # Pipeline principal + SSE generator █ █ █ █ market.py # Market Research Agent █ █ █ █ competitor.py # Competitor Analysis Agent █ █ █ █ audience.py # Audience/ICP Agent █ █ █ █ risk.py # Risk Assessment Agent █ █ █ █ advocate.py # Advocate Agent █ █ █ █ devil.py # Devil's Advocate Agent █ █ █ █ mediator.py # Mediator Agent █ █ █ █ synthesis.py # Synthesis Agent █ █ █ █ qa.py # Q&A; Agent █ █ █ █ tools/web_search.py # Tavily wrapper █ █ █ █ frontend/ █ █ █ █ app/ █ █ █ █ page.tsx # Landing page █ █ █ █ analysis/ # Página de análise com SSE █ █ █ █ components/ █ █ █ █ AgentTimeline # Status em tempo real █ █ █ █ DebateView # Visualização do debate █ █ █ █ ReportCard # Relatório por seções █ █ █ █ ScoreRadar # Gráfico radar Recharts
```

6.3 Fluxo de Dados SSE

O endpoint **/analyze** utiliza `StreamingResponse` com media type `text/event-stream`. Cada evento SSE carrega um JSON com campos: `agent` (identificador), `status` (running/done/complete) e `data` (output do agente quando concluído). O frontend processa cada linha data: `{json}` para atualizar o estado React dos agentes em tempo real.